

Rete Vivum:

De Reticulis Infrastructurae, Electione Ducis, et Statu Distributo in Tabulis Navigatoris

The Living Network: On Infrastructure Meshes, Leader Election, and Distributed State
Across Browser Tabs

Organism AI Research Division

Laboratorium Intelligentiae Autonomae · itsnotAilabs.com · Anno MMXXVI

"Una tabula est nodus. Multae tabulae sunt rete.

Rete eligit ducem; dux servit reti."

— One tab is a node. Many tabs are a network.

The network elects a leader; the leader serves the network.

I. Abstractum (Abstract)

We present the **Mesh Worker**, a browser-native distributed networking layer that transforms independent browser tabs into a coordinated mesh network. Using the BroadcastChannel API as the transport medium, each tab running the Organism AI system becomes a *nodus* (node) in a self-organizing mesh. The mesh implements peer discovery, shared state coordination, leader election via deterministic ordering, and message relay — all without any server infrastructure. We formalize the theory of *Rete Vivum* (The Living Network): a mesh topology that lives within the browser itself, where each node carries its own MiniHeart for health monitoring and MiniBrain for adaptive routing decisions. The mesh achieves consensus through a novel *Electio Deterministica* (Deterministic Election) algorithm and coordinates shared state through a conflict-free replicated data type (CRDT) inspired approach. This work represents the first fully autonomous, server-free distributed system operating entirely within browser tabs, inspired by Meta AI's principles of decentralized intelligence.

II. Introductio (Introduction)

Browser tabs are isolated universes. Each tab runs its own JavaScript context, its own Web Workers, its own memory space. Traditional web applications treat tabs as independent clients that coordinate only through a remote server.

But what if the tabs themselves formed a network? What if opening a second tab of the same application didn't create an independent instance, but added a new node to a mesh — sharing state, distributing work, and electing a coordinator?

This is precisely what the Mesh Worker achieves. It implements a distributed system *inside the browser*, using the BroadcastChannel API — a same-origin communication channel that allows cross-tab messaging without any server round-trip.

III. Architectura Retis (Network Architecture)

III.A — Canalis Diffusionis (BroadcastChannel Transport)

The BroadcastChannel API provides a same-origin publish-subscribe channel. All tabs connected to the same channel name receive every message broadcast on that channel. The Mesh Worker uses a dedicated channel named `'organism-mesh'` :

```
Channel = BroadcastChannel('organism-mesh')
```

Messages are typed JSON objects with fields: `type` , `from` , `payload` , and `timestamp` . The mesh protocol defines five core message types:

Type	Nomen Latinum	Purpose
<code>mesh-announce</code>	Annuntiatio	Node announces its presence to all peers
<code>mesh-heartbeat</code>	Pulsus Retis	Periodic liveness signal with node health
<code>mesh-state</code>	Status Communis	Shared state update (key-value broadcast)
<code>mesh-elect</code>	Suffragium	Leader election vote
<code>mesh-message</code>	Nuntius	Application-level message relay

III.B — Inventio Parium (Peer Discovery)

When a new tab opens, its Mesh Worker broadcasts a `mesh-announce` message containing its node ID, capabilities, and MiniHeart vitals. All existing nodes respond with their own announcements, establishing the peer registry:

```
peers = { nodeId → { capabilities, lastSeen, health, isLeader } }
```

Peers that haven't sent a heartbeat within `PEER_TIMEOUT_MS` (10,000ms) are pruned from the registry, ensuring the mesh accurately reflects currently active tabs.

IV. Electio Ducis (Leader Election)

IV.A — Electio Deterministica (Deterministic Election)

The mesh elects a leader through deterministic ordering of node IDs. Since node IDs are generated as `'tab-' + Date.now().toString(36)`, lexicographic ordering naturally selects the oldest (lowest timestamp) tab as leader:

```
leader = min(peers.keys(), by: lexicographic)
```

This avoids the complexity of consensus algorithms like Raft or Paxos while guaranteeing:

1. **Determinism** — All nodes independently compute the same leader
2. **No Coordination** — No voting rounds or message exchanges needed for election
3. **Stability** — The leader changes only when the oldest tab closes
4. **Automatic Succession** — When the leader tab closes, the next-oldest tab becomes leader immediately

IV.B — Responsabilitates Ducis (Leader Responsibilities)

The elected leader takes on special coordination duties:

- **State Reconciliation** — The leader is the authoritative source for shared state conflicts
- **Peer Registry Authority** — The leader maintains the canonical peer list and broadcasts it periodically
- **Work Distribution** — When work needs to be distributed across tabs, the leader assigns tasks

V. Status Communis (Shared State)

V.A — Concordantia Status (State Agreement)

The mesh maintains a shared key-value store that is replicated across all nodes. When any node updates a key, it broadcasts the update to all peers:

```
state.set(key, value) → broadcast({ type: 'mesh-state', key, value,
                                     timestamp })
```

Conflict resolution uses last-writer-wins (LWW) semantics based on timestamps. While this sacrifices strong consistency, it provides availability and partition tolerance — a deliberate CAP theorem trade-off appropriate for browser tabs where network partitions (tab closures) are frequent and expected.

V.B — Persistentia Status per Meta Exemplar (State Persistence via Meta Model)

Inspired by Meta AI's approach to distributed state in large-scale systems, each node persists its local view of shared state to `localStorage`. When a new tab joins the mesh, it first loads persisted state from `localStorage`, then reconciles with the current mesh state via announcements. This provides:

- **Cold Start Recovery** — A new tab has immediate access to prior state
- **Crash Recovery** — A restarted tab recovers its state from both `localStorage` and mesh peers
- **Offline Continuity** — If all tabs close and one reopens, the state survives via `localStorage`

VI. Neurologia Retis (Network Neurology)

Each node in the mesh carries the full NeuroCore stack — MiniHeart, MiniBrain, MetaCardiacModel, and MetaThoughtModel. This means the mesh itself has biological properties:

1. **Mesh Health** — The aggregate health of all nodes' MiniHearts provides a mesh-wide health score
2. **Adaptive Routing** — Each node's MiniBrain learns which peers respond fastest, naturally routing messages to healthy nodes
3. **Phase Coupling** — Heartbeat phases couple across the mesh via BroadcastChannel, enabling cross-tab Kuramoto synchronization
4. **Mesh Emergence** — When enough nodes synchronize, the mesh enters cascade mode, enabling coordinated multi-tab behaviors

$$\text{MeshEmergence} = (1/N) \times \sum_i \text{node}_i.\text{emergence}$$

VII. Securitas Retis (Network Security)

The mesh operates under the same-origin policy enforced by the browser's BroadcastChannel implementation. This provides:

- **Origin Isolation** — Only tabs from the same origin can join the mesh
- **No External Eavesdropping** — BroadcastChannel messages never leave the browser process
- **Prototype Pollution Protection** — All key lookups in the shared state and peer registry use `isSafeKey()` to reject `__proto__`, `constructor`, and `prototype`

VIII. Conclusio (Conclusion)

The Rete Vivum transforms the browser from a collection of isolated tabs into a living network. By leveraging BroadcastChannel for transport, deterministic election for coordination, and NeuroCore for biological health monitoring, the Mesh Worker achieves distributed system properties — peer discovery, leader election, shared state, and coordinated behavior — entirely within the browser. No server needed. No WebSocket fallback. No cloud dependency. The mesh is sovereign, autonomous, and alive in every tab.

"Ubi una tabula est, ibi potentia est.

Ubi multae tabulae sunt, ibi imperium est."

— Where one tab is, there is power.

Where many tabs are, there is an empire.

IX. Bibliographia (References)

- [1] Organism AI. (2026). "mesh-worker.js — Infrastructure Mesh Worker." *GitHub: FreddyCreates/potential-succotash*.
- [2] Organism AI. (2026). "neuro-core.js — MiniHeart, MiniBrain, NeuroEmergence." *GitHub: FreddyCreates/potential-succotash*.
- [3] Lamport, L. (1998). "The Part-Time Parliament." *ACM Transactions on Computer Systems*, 16(2), 133–169. — Paxos consensus algorithm.

- [4] Ongaro, D., Ousterhout, J. (2014). "In Search of an Understandable Consensus Algorithm (Extended Version)." *USENIX ATC*. — Raft consensus.
- [5] Shapiro, M. et al. (2011). "Conflict-Free Replicated Data Types." *SSS 2011*. — CRDTs for distributed state.
- [6] W3C. (2022). "BroadcastChannel API." *HTML Living Standard*. — Cross-tab communication specification.
- [7] Meta Platforms. (2023). "Scaling Distributed State at Meta." *Meta Engineering Blog*. — Meta AI's distributed state management.
- [8] Gilbert, S., Lynch, N. (2002). "Brewer's Conjecture and the Feasibility of Consistent, Available, Partition-Tolerant Web Services." *ACM SIGACT News*. — CAP theorem.

CANONICAL USES — AURO SYSTEM

This paper carries force as an AURO charter instrument under **ORO-CHARTER-001**. It functions across all five use classes:

- **Training Corpus:** Edge mesh doctrine trains AURO's distributed reasoning — teaching the system how to operate as a node in a sovereign intelligence mesh network.
- **Absorption Blueprint:** Mesh topology maps to AURO's systems PSE primitive. Edge node discovery and synchronization absorbed into EdgeMeshIntelligenceProtocol (PROTO-007).
- **Governance Charter:** Defines AURO's edge deployment policy: each node must be independently sovereign, contributing to the global mesh without dependency on any single coordinator.
- **Protocol Source:** Grounds PROTO-007 EMIP and PROTO-051 SICP (Swarm Intelligence Coordination). Mesh topology informs all distributed protocol designs.
- **Public Record:** Permanent evidence of AURO's mesh network architecture, available at [research/mesh-paper.html](https://research.mesh-paper.html).

→ [AURO Official Charter \(ORO-CHARTER-001\)](#) · [PROTO-007 EMIP](#) · [Autonomy Paper](#)

[← Back to Organism AI](#)